

A Voyage to the Kernel



Part 21

Segment: 4.1, Day 20

In Segment 3, we'd looked at some fundamental aspects of Linux kernel module programming and we discussed commands like *lsmod* and files like */etc/modprobe.conf*. We also meddled with `__init` and `__exit` while writing a simple module. This new segment is dedicated to kernel programming, in general.

The easiest way to add code to the kernel is to just add the lines to the source tree and recompile it. You may recall that you can actually pre-configure the kernel you are about to compile (please refer to the previous segment for more details).

We have also seen that we can add some code to the 'kernel' even when it is alive. This is what we did when we wrote simple kernel modules (or loadable kernel modules) and loaded them to the kernel. Typically, if you just want to write a driver for new hardware that you have made, and enable the device, then you don't have to compile the entire kernel to make the device run. A module will serve the purpose. So is the case with system calls. This is one of the properties of the Linux kernel. Well, for our discussions, I'm going to call the kernel that gets loaded while booting up as the 'base kernel' and loadable ones as modules (a generic term).

Module programming became popular during the mid-90s and attracted many developers (and vendors). But the earlier modules were not that easily 'loadable'. It was only by the beginning of this decade that they attained the shape of modules, as we see them today.

Why should it be loadable?

This is one of the fundamental questions that beginners ask. "Why should I write a loadable module? Why not incorporate it into the kernel code itself?" Well, you may find some reasons if you think of yourself as a hardware vendor who's going to offer a new piece of hardware. As discussed before, the users' convenience is a factor. They don't have to rebuild the kernel each time they add a new device driver.

Also, if you try to rebuild the kernel with your code, then it may be hard for the developers to fix the error that may pop up during the process. Sometimes the error may even prevent you from logging into the system if you have a serious bug in the code you just added (since it will be executed during the boot stage).

If you look at the resource side, then you can see that the module saves much of your memory, as you will be loading it only when it is needed. Another fact is that developers (say, hardware vendors) find it easy to maintain their code if it is written as a module.

This is even helpful during the development phase. You can experiment with your code very easily if you write it as a module. It will save you time as you don't have to reboot the system each time you update the code. But you also need to note that if you are writing a driver for a device that is essential for providing the service, then you should try to incorporate the code into the base kernel itself.

Useful utilities

While developing the module, you may find the following utilities quite handy:

- *insmod* – to load a module into the kernel (refer to the previous segment—for your information, I'm putting the *moduleloader* header file code here):

```
#ifndef _LINUX_MODULELOADER_H
#define _LINUX_MODULELOADER_H
#include <linux/module.h>
#include <linux/elf.h>
int module_frob_arch_sections(Elf_Shdr *hdr,
                             Elf_Shdr *sechdrs,
                             char *secstrings,
                             struct module *mod);
unsigned int arch_mod_section_prepend(struct module *mod,
unsigned int section);
void *module_alloc(unsigned long size);
void module_free(struct module *mod, void *module_region);
int apply_relocate(Elf_Shdr *sechdrs,
                  const char *strtab,
                  unsigned int symindex,
```